# Practice · if and else

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

## (mc) L1 · The shape of an if

Which line starts an `if` correctly?

- `if score >= 50 {`
- `if (score >= 50);`
- **Right answer:** `if (score >= 50) {`
- `if [score >= 50] {`

*Feedback:* The condition goes inside round brackets, and the block of lines to run goes inside curly braces `{ }`. No semicolon after the closing round bracket: a `;` there would end the `if` with an empty block.

## (mc) L1 · What else is for

When does the `else` block run?

- Always, after the `if` block.
- **Right answer:** Only when the `if` condition is false.
- Only when the `if` condition is true.
- Never. `else` is just a label.

*Feedback:* `if` and `else` are two doors and exactly one of them opens. Condition true: the `if` block runs. Condition false: the `else` block runs. Never both, never neither (slide 12).

## (mc) L1 · What the braces hold

```
if (x > 0) {
    printf("positive\n");
    printf("checked\n");
}
```

When `x` is `-3`, what is printed?

- `positive` and `checked`
- `checked` only
- **Right answer:** Nothing.
- `positive` only

*Feedback:* Both `printf` lines are inside the braces, so both belong to the `if`. The condition `-3 > 0` is false, so the whole block is skipped and nothing is printed.

## (mc) L2 · if without else

```
int score;
printf("Score: ");
scanf("%d", &score);
if (score >= 50) {
    printf("Passed\n");
}
```

```
printf("Done\n");
```

You type `49`. What is printed after `Score:` ?

- `Passed` and `Done`
- **Right answer:** `Done` only
- Nothing.
- `Passed` only

*Feedback:* `49 >= 50` is false, so the `if` block is skipped. `printf("Done\n")` is **outside** the braces, so it runs no matter what. With `50` you would see `Passed` and then `Done`.

## (mc) L2 · else-if chain, top to bottom

```
if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else if (score >= 70) {
    printf("C\n");
} else {
    printf("Needs improvement\n");
}
```

`score` is `95`. What is printed?

- **Right answer:** `A`
- `A`, `B` and `C`
- `C`
- `Needs improvement`

*Feedback:* C checks from the top. `95 >= 90` is true, so `A` is printed and the **rest of the chain is skipped**. It does not go on to check 80 and 70. Only one branch of a chain ever runs.

## (mc) L2 · The wrong order

```
if (score >= 70) {
    printf("C\n");
} else if (score >= 80) {
    printf("B\n");
} else if (score >= 90) {
    printf("A\n");
}
```

`score` is `95`. What is printed?

- `A`
- `A`, `B` and `C`
- **Right answer:** `C`
- Nothing. The compiler refuses this order.

*Feedback:* The program compiles fine, but it is wrong. `95 >= 70` is already true, so the first branch prints `C` and the others are never reached. Nobody can ever get `A` or `B` with this order. Slide 14's fix: check the biggest threshold first, 90, then 80, then 70.

## (fib) L3 · The middle of a chain

Complete the keywords that start the second branch. Type the two words with one space between them.

```
if (score >= 90) {
    printf("A\n");
} [else if] (score >= 80) {
    printf("B\n");
} else {
    printf("C\n");
}
```

*Feedback:* `else if` starts a branch that is tried only when the branches above it were false. It is two words in C, not one.

## (fib) L3 · The missing brace

This program will not compile. The compiler says `expected '}' before 'else'`. Type the one missing character.

```
if (age >= 18) {
    printf("Adult ticket\n");
[}] else {
    printf("Student ticket\n");
}
```

*Feedback:* Every `{` needs a `}`. The `if` block was opened but never closed, so the compiler met `else` while still inside the `if`. Count your braces when you see `expected '}'`.

## (fib) L3 · The condition for a free ticket

Complete the condition so that a child under 6 gets a free ticket. Type it with single spaces, like `age < 6`.

```
if ([age < 6|age<6|age <= 5|age<=5]) {
    printf("Free ticket\n");
} else {
    printf("Normal ticket\n");
}
```

*Feedback:* "Under 6" means 5 or less, so `age < 6` (or `age <= 5`). `age < 5` would make a five-year-old pay, and `age <= 6` would let a six-year-old in free.